```c
//###########################################################################
// FILE:   LABstarter_main.c
//
// TITLE:  Lab Starter
//###########################################################################

// Included Files
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>
#include <math.h>
#include <limits.h>
#include "F28x_Project.h"
#include "driverlib.h"
#include "device.h"
#include "F28379dSerial.h"
#include "song.h"
#include "dsp.h"
#include "fpu32/fpu_rfft.h"

#define PI          3.14159265358979323846264338327795
#define TWOPI       6.28318530717958647692528676655900
#define HALFPI      1.57079632679489661923132169163980
// The Launchpad's CPU Frequency set to 200 you should not change this value
#define LAUNCHPAD_CPU_FREQUENCY 200


// Interrupt Service Routines predefinition
__interrupt void cpu_timer0_isr(void);
__interrupt void cpu_timer1_isr(void);
__interrupt void cpu_timer2_isr(void);
__interrupt void SWI_isr(void);

// sjew: cus interrupt
__interrupt void ADCD_ISR (void);
__interrupt void ADCC_ISR (void);
__interrupt void ADCB_ISR (void);

// Count variables
uint32_t numTimer0calls = 0;
uint32_t numSWIcalls = 0;
extern uint32_t numRXA;
uint16_t UARTPrint = 0;

// sjew: custom functions
void setDACA(float volt);

// sjew: cus variables
float adcd1result_volts = 0;
float adcd0result_volts = 0;
uint32_t ADCD0_count = 0;

// sjew: adc isr variables
uint32_t ADCC_count = 0;

float adcc2result_volts = 0;
```

```
 58 float adcc3result_volts = 0;
 59 float adcc4result_volts = 0;
 60 float adcc5result_volts = 0;
 61
 62 // sjew: initializations for IMU data
 63 float IMUx = 0;
 64 float IMU4x = 0;
 65 float IMUz = 0;
 66 float IMU4z = 0;
 67 // sjew: start the offsets at 1.23 because that is what they are assumed to be but then theyre
    adjusted later
 68 float IMU_x_offset = 1.23;
 69 float IMU_4x_offset = 1.23;
 70 float IMU_z_offset = 1.23;
 71 float IMU_4z_offset = 1.23;
 72
 73 float sum_x = 0;
 74 float sum_4x = 0;
 75 float sum_z = 0;
 76 float sum_4z = 0;
 77
 78 uint8_t adcc_count_flag = 0;
 79 // adcb values
 80 float adcb2_volts = 0;
 81
 82 //xk is the current ADC reading, xk_1 is the ADC reading one millisecond ago, xk_2 two
    milliseconds ago, etc
 83 float xk_arr[22] = {
 84                     0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
 85 };
 86 //yk is the filtered value
 87 float yk = 0;
 88 //b is the filter coefficients
 89 float b[22]={   -2.3890045153263611e-03,
 90                 -3.3150057635348224e-03,
 91                 -4.6136191242627002e-03,
 92                 -4.1659855521681268e-03,
 93                 1.4477422497795286e-03,
 94                 1.5489414225159667e-02,
 95                 3.9247886844071371e-02,
 96                 7.0723964095458614e-02,
 97                 1.0453473887246176e-01,
 98                 1.3325672639406205e-01,
 99                 1.4978314227429904e-01,
100                 1.4978314227429904e-01,
101                 1.3325672639406205e-01,
102                 1.0453473887246176e-01,
103                 7.0723964095458614e-02,
104                 3.9247886844071371e-02,
105                 1.5489414225159667e-02,
106                 1.4477422497795286e-03,
107                 -4.1659855521681268e-03,
108                 -4.6136191242627002e-03,
109                 -3.3150057635348224e-03,
110                 -2.3890045153263611e-03};
111
112 void main(void)
```

```
113 {
114     // PLL, WatchDog, enable Peripheral Clocks
115     // This example function is found in the F2837xD_SysCtrl.c file.
116     InitSysCtrl();
117
118     InitGpio();
119
120     // Blue LED on LaunchPad
121     GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
122     GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
123     GpioDataRegs.GPASET.bit.GPIO31 = 1;
124
125     // Red LED on LaunchPad
126     GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
127     GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
128     GpioDataRegs.GPBSET.bit.GPIO34 = 1;
129
130     // LED1 and PWM Pin
131     GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 0);
132     GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);
133     GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
134
135     // LED2
136     GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
137     GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
138     GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
139
140     // LED3
141     GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
142     GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
143     GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
144
145     // LED4
146     GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
147     GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
148     GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
149
150     // LED5
151     GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
152     GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
153     GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
154
155     // LS7366#1 CS
156     GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
157     GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
158     GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;
159
160     // LS7366#2 CS
161     GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
162     GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
163     GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;
164
165     // LS7366#3 CS
166     GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);
167     GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
168     GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;
169
```

```
170     // LS7366#4 CS
171     GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
172     GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
173     GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;
174
175     // WIZNET RST
176     GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
177     GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
178     GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;
179
180     //PushButton 1
181     GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
182     GPIO_SetupPinOptions(157, GPIO_INPUT, GPIO_PULLUP);
183
184     //PushButton 2
185     GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
186     GPIO_SetupPinOptions(158, GPIO_INPUT, GPIO_PULLUP);
187
188     //PushButton 3
189     GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
190     GPIO_SetupPinOptions(159, GPIO_INPUT, GPIO_PULLUP);
191
192     //PushButton 4
193     GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
194     GPIO_SetupPinOptions(160, GPIO_INPUT, GPIO_PULLUP);
195
196     //SPIRAM  CS  Chip Select
197     GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
198     GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSHPULL);
199     GpioDataRegs.GPASET.bit.GPIO19 = 1;
200
201     //F28027 CS
202     GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
203     GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSHPULL);
204     GpioDataRegs.GPASET.bit.GPIO29 = 1;
205
206     //MPU9250  CS  Chip Select
207     GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
208     GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL);
209     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
210
211     //WIZNET  CS  Chip Select
212     GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
213     GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSHPULL);
214     GpioDataRegs.GPDSET.bit.GPIO125 = 1;
215
216     // Clear all interrupts and initialize PIE vector table:
217     // Disable CPU interrupts
218     DINT;
219
220     // Initialize the PIE control registers to their default state.
221     // The default state is all PIE interrupts disabled and flags
222     // are cleared.
223     // This function is found in the F2837xD_PieCtrl.c file.
224     InitPieCtrl();
225
226     // Disable CPU interrupts and clear all CPU interrupt flags:
```

```
227     IER = 0x0000;
228     IFR = 0x0000;
229
230     // Initialize the PIE vector table with pointers to the shell Interrupt
231     // Service Routines (ISR).
232     // This will populate the entire table, even if the interrupt
233     // is not used in this example.  This is useful for debug purposes.
234     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
235     // This function is found in F2837xD_PieVect.c.
236     InitPieVectTable();
237
238     // Interrupts that are used in this example are re-mapped to
239     // ISR functions found within this project
240     EALLOW;  // This is needed to write to EALLOW protected registers
241     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
242     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
243     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
244     PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
245     PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
246     PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
247     PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
248     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
249     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
250     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
251     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
252
253     //adc isr
254     PieVectTable.ADCD1_INT = &ADCD_ISR;
255
256     PieVectTable.ADCC1_INT = &ADCC_ISR;
257
258     PieVectTable.ADCB1_INT = &ADCB_ISR;
259
260     PieVectTable.EMIF_ERROR_INT = &SWI_isr;
261     EDIS;    // This is needed to disable write to EALLOW protected registers
262
263
264     // Initialize the CpuTimers Device Peripheral. This function can be
265     // found in F2837xD_CpuTimers.c
266     InitCpuTimers();
267
268     // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
269     // 200MHz CPU Freq,                     Period (in uSeconds)
270     ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);
271     ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 20000);
272     ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000);
273
274     // Enable CpuTimer Interrupt bit TIE
275     CpuTimer0Regs.TCR.all = 0x4000;
276     CpuTimer1Regs.TCR.all = 0x4000;
277     CpuTimer2Regs.TCR.all = 0x4000;
278
279     init_serialSCIA(&SerialA,115200);
280     init_serialSCIB(&SerialB,19200);
281 //    init_serialSCIC(&SerialC,115200);
282 //    init_serialSCID(&SerialD,115200);
283
```

```
284     EALLOW;
285     // sjew: epwm trigger for sampling ADC at a 1ms rate
286     EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
287     EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
288     EPwm4Regs.ETSEL.bit.SOCASEL = 0x2; // Select Event when counter equal to PRD
289     EPwm4Regs.ETPS.bit.SOCAPRD = 0x1; // Generate pulse on 1st event (“pulse” is the same as
    “trigger”)
290     EPwm4Regs.TBCTR = 0x0; // Clear counter
291     EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
292     EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
293     EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
294     EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
295     // Notice here that we are not setting CMPA or CMPB because we are not using the PWM
    signal
296     EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
297     EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
298
299     // sjew: second ADC trigger because microphone is sampled at a 0.1ms rate
300     EPwm7Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
301     EPwm7Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
302     EPwm7Regs.ETSEL.bit.SOCASEL = 0x2; // Select Event when counter equal to PRD
303     EPwm7Regs.ETPS.bit.SOCAPRD = 0x1; // Generate pulse on 1st event (“pulse” is the same as
    “trigger”)
304     EPwm7Regs.TBCTR = 0x0; // Clear counter
305     EPwm7Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
306     EPwm7Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
307     EPwm7Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
308     EPwm7Regs.TBPRD = 5000; // Set Period to 0.1ms sample. Input clock is 50MHz.
309     // Notice here that we are not setting CMPA or CMPB because we are not using the PWM
    signal
310     EPwm7Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
311     EPwm7Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
312     EDIS;
313
314     EALLOW;
315     //write configurations for all ADCs ADCA, ADCB, ADCC, ADCD
316     AdcaRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4
317     AdcbRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4
318     AdccRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4
319     AdcdRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4
320     AdcSetMode(ADC_ADCA, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration
    settings
321     AdcSetMode(ADC_ADCB, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration
    settings
322     AdcSetMode(ADC_ADCC, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration
    settings
323     AdcSetMode(ADC_ADCD, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration
    settings
324     //Set pulse positions to late
325     AdcaRegs.ADCCTL1.bit.INTPULSEPOS = 1;
326     AdcbRegs.ADCCTL1.bit.INTPULSEPOS = 1;
327     AdccRegs.ADCCTL1.bit.INTPULSEPOS = 1;
328     AdcdRegs.ADCCTL1.bit.INTPULSEPOS = 1;
329
330     //power up the ADCs
331     AdcaRegs.ADCCTL1.bit.ADCPWDNZ = 1;
332     AdcbRegs.ADCCTL1.bit.ADCPWDNZ = 1;
```

```
333     AdccRegs.ADCCTL1.bit.ADCPWDNZ = 1;
334     AdcdRegs.ADCCTL1.bit.ADCPWDNZ = 1;
335     //delay for 1ms to allow ADC time to power up
336     DELAY_US(1000);
337     //Select the channels to convert and end of conversion flag
338     //Many statements commented out, To be used when using ADCA or ADCB
339     //ADCA
340     //AdcaRegs.ADCSOC0CTL.bit.CHSEL = ???; //SOC0 will convert Channel you choose Does not
    have to be A0
341     //AdcaRegs.ADCSOC0CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
342     //AdcaRegs.ADCSOC0CTL.bit.TRIGSEL = ???;// EPWM4 ADCSOCA or another trigger you choose
    will trigger SOC0
343     //AdcaRegs.ADCSOC1CTL.bit.CHSEL = ???; //SOC1 will convert Channel you choose Does not
    have to be A1
344     //AdcaRegs.ADCSOC1CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
345     //AdcaRegs.ADCSOC1CTL.bit.TRIGSEL = ???;// EPWM4 ADCSOCA or another trigger you choose
    will trigger SOC1
346     //AdcaRegs.ADCINTSEL1N2.bit.INT1SEL = ???; //set to last SOC that is converted and it will
    set INT1 flag ADCA1
347     //AdcaRegs.ADCINTSEL1N2.bit.INT1E = 1; //enable INT1 flag
348     //AdcaRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //make sure INT1 flag is cleared
349     //ADCB Microphone is connected to ADCINB4
350     AdcbRegs.ADCSOC0CTL.bit.CHSEL = 0x4; //SOC0 will convert Channel you choose Does not have
    to be B0
351     AdcbRegs.ADCSOC0CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
352     AdcbRegs.ADCSOC0CTL.bit.TRIGSEL = 0x11;// EPWM7 ADCSOCA or another trigger you choose will
    trigger SOC0
353 //    AdcbRegs.ADCSOC1CTL.bit.CHSEL = ???; //SOC1 will convert Channel you choose Does not
    have to be B1
354 //    AdcbRegs.ADCSOC1CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
355 //    AdcbRegs.ADCSOC1CTL.bit.TRIGSEL = ???;// EPWM7 ADCSOCA or another trigger you choose
    will trigger SOC1
356     AdcbRegs.ADCINTSEL1N2.bit.INT1SEL = 0x0; //set to last SOC that is converted and it will
    set INT1 flag ADCB1
357     AdcbRegs.ADCINTSEL1N2.bit.INT1E = 1; //enable INT1 flag
358     AdcbRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //make sure INT1 flag is cleared
359     //ADCC
360     AdccRegs.ADCSOC0CTL.bit.CHSEL = 0x2; //SOC0 will convert Channel you choose Does not have
    to be B0
361     AdccRegs.ADCSOC0CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
362     AdccRegs.ADCSOC0CTL.bit.TRIGSEL = 0xb;// EPWM4 ADCSOCA or another trigger you choose will
    trigger SOC0
363     AdccRegs.ADCSOC1CTL.bit.CHSEL = 0x3; //SOC1 will convert Channel you choose Does not have
    to be B1
364     AdccRegs.ADCSOC1CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
365     AdccRegs.ADCSOC1CTL.bit.TRIGSEL = 0xb;// EPWM4 ADCSOCA or another trigger you choose will
    trigger SOC1
366     AdccRegs.ADCSOC2CTL.bit.CHSEL = 0x4; //SOC2 will convert Channel you choose Does not have
    to be B2
367     AdccRegs.ADCSOC2CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
368     AdccRegs.ADCSOC2CTL.bit.TRIGSEL = 0xb;// EPWM4 ADCSOCA or another trigger you choose will
    trigger SOC2
369     AdccRegs.ADCSOC3CTL.bit.CHSEL = 0x5; //SOC3 will convert Channel you choose Does not have
    to be B3
370     AdccRegs.ADCSOC3CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
371     AdccRegs.ADCSOC3CTL.bit.TRIGSEL = 0xb;// EPWM4 ADCSOCA or another trigger you choose will
    trigger SOC3
```

```
372     AdccRegs.ADCINTSEL1N2.bit.INT1SEL = 0x3; //set to last SOC that is converted and it will
    set INT1 flag ADCB1
373     AdccRegs.ADCINTSEL1N2.bit.INT1E = 1; //enable INT1 flag
374     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //make sure INT1 flag is cleared
375     //ADCD
376
377     AdcdRegs.ADCSOC0CTL.bit.CHSEL = 0; // set SOC0 to convert pin D0
378     AdcdRegs.ADCSOC0CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
379     AdcdRegs.ADCSOC0CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA will trigger SOC0
380     AdcdRegs.ADCSOC1CTL.bit.CHSEL = 1; //set SOC1 to convert pin D1
381     AdcdRegs.ADCSOC1CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
382     AdcdRegs.ADCSOC1CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA will trigger SOC1
383     //AdcdRegs.ADCSOC2CTL.bit.CHSEL = ???; //set SOC2 to convert pin D2
384     //AdcdRegs.ADCSOC2CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
385     //AdcdRegs.ADCSOC2CTL.bit.TRIGSEL = ???; // EPWM4 ADCSOCA will trigger SOC2
386     //AdcdRegs.ADCSOC3CTL.bit.CHSEL = ???; //set SOC3 to convert pin D3
387     //AdcdRegs.ADCSOC3CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
388     //AdcdRegs.ADCSOC3CTL.bit.TRIGSEL = ???; // EPWM4 ADCSOCA will trigger SOC3
389     AdcdRegs.ADCINTSEL1N2.bit.INT1SEL = 1; //set to SOC1, the last converted, and it will set
    INT1 flag ADCD1
390     AdcdRegs.ADCINTSEL1N2.bit.INT1E = 1; //enable INT1 flag
391     AdcdRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //make sure INT1 flag is cleared
392     EDIS;
393
394     // Enable DACA and DACB outputs
395     EALLOW;
396     DacaRegs.DACOUTEN.bit.DACOUTEN = 1;//enable dacA output-->uses ADCINA0
397     DacaRegs.DACCTL.bit.LOADMODE = 0;//load on next sysclk
398     DacaRegs.DACCTL.bit.DACREFSEL = 1;//use ADC VREF as reference voltage
399     DacbRegs.DACOUTEN.bit.DACOUTEN = 1;//enable dacB output-->uses ADCINA1
400     DacbRegs.DACCTL.bit.LOADMODE = 0;//load on next sysclk
401     DacbRegs.DACCTL.bit.DACREFSEL = 1;//use ADC VREF as reference voltage
402     EDIS;
403
404     // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
405     // which is connected to CPU-Timer 1, and CPU int 14, which is connected
406     // to CPU-Timer 2:  int 12 is for the SWI.
407     IER |= M_INT1;
408     IER |= M_INT8;  // SCIC SCID
409     IER |= M_INT9;  // SCIA
410     IER |= M_INT12;
411     IER |= M_INT13;
412     IER |= M_INT14;
413
414     // Enable TINT0 in the PIE: Group 1 interrupt 7
415     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
416     // Enable SWI in the PIE: Group 12 interrupt 9
417     PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
418
419     // sjew: adc interrupt enable
420     PieCtrlRegs.PIEIER1.bit.INTx6 = 1;
421     PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
422     PieCtrlRegs.PIEIER1.bit.INTx2 = 1;
423
424     // Enable global Interrupts and higher priority real-time debug events
425     EINT;  // Enable Global interrupt INTM
426     ERTM;  // Enable Global realtime interrupt DBGM
```

```
427
428
429     // IDLE loop. Just sit and loop forever (optional):
430     while(1)
431     {
432         if (UARTPrint == 1 ) {
433             // Normally on the Robot Car we only use the below UART_printfLine functions to
    write to the
434             // on board LCD screen.  This below serial_printf is only used in lab 1 to print
    to a serial
435             // terminal over a USB cable like you will for your Homeworks.
436             //serial_printf(&SerialA,"Num Timer2:%ld Num SerialRX: %ld\r
    \n",CpuTimer2.InterruptCount,numRXA);
437
438             //IMPORTANT!! %ld is for an int32_t.  To print an int16_t use %d
439             UART_printfLine(1,"x: %.2f,4x: %.2f", IMUx, IMU4x);
440             UART_printfLine(2,"z: %.2f,4z: %.2f", IMUz, IMU4z);
441             UARTPrint = 0;
442         }
443     }
444 }
445
446
447 // SWI_isr,  Using this interrupt as a Software started interrupt
448 __interrupt void SWI_isr(void) {
449
450     // These three lines of code allow SWI_isr, to be interrupted by other interrupt functions
451     // making it lower priority than all other Hardware interrupts.
452     PieCtrlRegs.PIEACK.all = PIEACK_GROUP12;
453     asm("       NOP");                  // Wait one cycle
454     EINT;                               // Clear INTM to enable interrupts
455
456
457
458     // Insert SWI ISR Code here.......
459
460
461     numSWIcalls++;
462
463     DINT;
464
465 }
466
467 // cpu_timer0_isr - CPU Timer0 ISR
468 __interrupt void cpu_timer0_isr(void)
469 {
470     CpuTimer0.InterruptCount++;
471
472     numTimer0calls++;
473
474 //    if ((numTimer0calls%50) == 0) {
475 //        PieCtrlRegs.PIEIFR12.bit.INTx9 = 1;  // Manually cause the interrupt for the SWI
476 //    }
477
478     if ((numTimer0calls%5) == 0) {
479         // Blink LaunchPad Red LED
480         GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
```

```
481     }
482
483
484     // Acknowledge this interrupt to receive more interrupts from group 1
485     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
486 }
487
488 // cpu_timer1_isr - CPU Timer1 ISR
489 __interrupt void cpu_timer1_isr(void)
490 {
491
492     CpuTimer1.InterruptCount++;
493 }
494
495 // cpu_timer2_isr CPU Timer2 ISR
496 __interrupt void cpu_timer2_isr(void)
497 {
498     // Blink LaunchPad Blue LED
499     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
500
501     CpuTimer2.InterruptCount++;
502
503 //  if ((CpuTimer2.InterruptCount % 10) == 0) {
504 //      UARTPrint = 1;
505 //  }
506 }
507
508 //This function sets DACA to the voltage between 0V and 3V passed to this function.
509 //If outside 0V to 3V the output is saturated at 0V to 3V
510 //Example code
511 //float myu = 2.25;
512 //setDACA(myu); // DACA will now output 2.25 Volts
513 void setDACA(float dacouta0) {
514     int16_t DACOutInt = 0;
515     // sjew: convert voltage value to a digital value, constrain to range and then set the
    digital value to the DAC to output
516     DACOutInt = dacouta0 * 4095.0 / 3.0; // perform scaling of 0 – almost 3.0V to 0 – 4095
517     if (DACOutInt > 4095) DACOutInt = 4095;
518     if (DACOutInt < 0) DACOutInt = 0;
519     DacaRegs.DACVALS.bit.DACVALS = DACOutInt;
520 }
521
522 void setDACB(float dacouta1) {
523     int16_t DACOutInt = 0;
524     // sjew: convert voltage value to a digital value, constrain to range and then set the
    digital value to the DAC to output
525     DACOutInt =  dacouta1 * 4095.0 / 3.0; // perform scaling of 0 – almost 3V to 0 – 4095
526     if (DACOutInt > 4095) DACOutInt = 4095;
527     if (DACOutInt < 0) DACOutInt = 0;
528     DacbRegs.DACVALS.bit.DACVALS = DACOutInt;
529 }
530
531 //adcd1 pie interrupt
532 __interrupt void ADCD_ISR (void)
533 {
534     // sjew: get the adcd 0 and 1 values and convert to volts
535     int16_t adcd0result = AdcdResultRegs.ADCRESULT0;
```

```
536     int16_t adcd1result = AdcdResultRegs.ADCRESULT1;
537
538     // Here covert ADCIND0 to volts
539     adcd0result_volts = adcd0result * 3.0 / 4095.0;
540     adcd1result_volts = adcd1result * 3.0 / 4095.0;
541
542     // Here covert ADCIND0, ADCIND1 to volts
543     // sjew: update the array to represent the 20 most recent values including the current
    value
544     float prev = adcd0result_volts;
545     for(int i = 0; i < 22; i++) {
546         float tmp = xk_arr[i];
547         xk_arr[i] = prev;
548         prev = tmp;
549     }
550     yk = 0;
551     // populate the yk value by dot product of coefficients and 22 most recent values
552     for(int i = 0; i < 22; i++) {
553         yk += b[i]*xk_arr[i];
554     }
555     // sjew: set DACA
556     // Here write voltages value to DACA
557 //    setDACA(yk);
558
559     // Print ADCIND0's voltage value to LCD every 100ms by setting UARTPrint =1
560 //    if ((ADCD0_count % 100) == 0) {
561 //        UARTPrint = 1;
562 //    }
563 //    ADCD0_count++;
564
565
566     AdcdRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
567     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
568
569 }
570
571 __interrupt void ADCC_ISR (void){
572     // sjew: get the z, 4z, x, 4x adc values from the gyro and convert to voltages
573     int16_t adcc2result = AdccResultRegs.ADCRESULT0;
574     int16_t adcc3result = AdccResultRegs.ADCRESULT1;
575     int16_t adcc4result = AdccResultRegs.ADCRESULT2;
576     int16_t adcc5result = AdccResultRegs.ADCRESULT3;
577
578     adcc2result_volts = adcc2result * 3.0 / 4095.0;
579     adcc3result_volts = adcc3result * 3.0 / 4095.0;
580     adcc4result_volts = adcc4result * 3.0 / 4095.0;
581     adcc5result_volts = adcc5result * 3.0 / 4095.0;
582     // sjew: convert voltages to deg/sec using the offset
583     IMUx = (adcc2result_volts - IMU_x_offset ) * 400.0;
584     IMU4x = (adcc3result_volts - IMU_4x_offset ) * 100.0;
585
586     IMUz = (adcc5result_volts - IMU_z_offset ) * 400.0;
587     IMU4z = (adcc4result_volts - IMU_4z_offset ) * 100.0;
588     // sjew: print every 500*1ms
589     if ((ADCC_count % 500) == 0) {
590         UARTPrint = 1;
591     }
```

```
592     //sjew: if the count is between 1000*1ms and 3000*1ms (1-3s) and the first time its
    reached these values
593     //      then sum the current voltage output from the DAC to take the average
594     if(ADCC_count > 1000 && ADCC_count <= 3000 && adcc_count_flag == 0) {
595         sum_x += adcc2result_volts;
596         sum_4x += adcc3result_volts;
597         sum_z += adcc5result_volts;
598         sum_4z += adcc4result_volts;
599     } else if(ADCC_count > 3000 && adcc_count_flag == 0) {
600         // sjew: after the first three seconds, take the average of all the offsets and set
    the count flag to 1 ( completed )
601         adcc_count_flag = 1;
602         IMU_z_offset = sum_z / 2000.0;
603         IMU_4z_offset = sum_4z / 2000.0;
604         IMU_x_offset = sum_x / 2000.0;
605         IMU_4x_offset = sum_4x / 2000.0;
606     }
607     ADCC_count++;
608
609     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
610     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
611 }
612
613
614 __interrupt void ADCB_ISR (void) {
615     // sjew: get the adc result from the register and convert it to a voltage value
616     int16_t adcb2result = AdcbResultRegs.ADCRESULT0;
617     adcb2_volts = adcb2result * 3.0 / 4095.0;
618     // sjew: set the dac for scope output
619     setDACA(adcb2_volts);
620
621
622     AdcbRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
623     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
624 }
625
626
```